

Spark SQL

源码深度分析

核心流程图 · 类设计说明 · 架构解析

基于 Apache Spark v3.3.1 源码

涵盖: catalyst · core · hive · connect · api · pipelines

目录

第一章	Spark SQL 模块总览架构
第二章	端到端查询执行流程
第三章	Catalyst 优化框架核心设计
第四章	QueryExecution 执行阶段详解
第五章	物理计划生成与 Join 策略选择
第六章	WholeStageCodegen 全阶段代码生成
第七章	AQE 自适应查询执行
第八章	Spark Connect 远程架构
第九章	Hive 集成与 Pipelines 模块
第十章	核心类设计总结
第十一章	总结与架构评价

第一章 Spark SQL 模块总览架构

Spark SQL 是 Apache Spark 最核心的模块之一，提供了从 SQL 解析到分布式执行的完整查询处理能力。整个 sql/ 目录包含 7 个子模块，各司其职又紧密协作，形成了层次分明、高度可扩展的查询引擎架构。

1.1 模块架构总览图

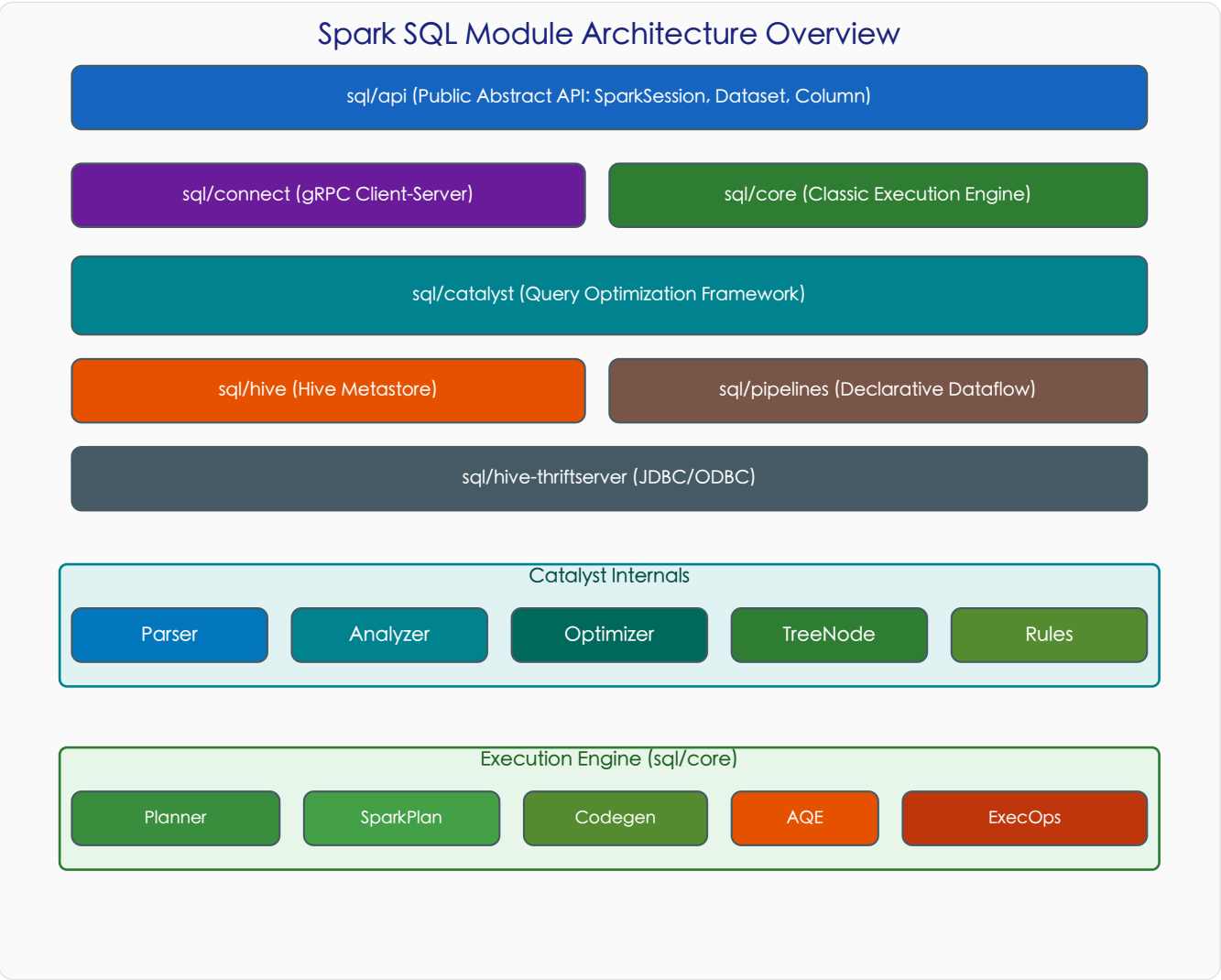


图 1-1: Spark SQL 模块架构总览（层次依赖关系）

1.2 各模块功能定位

模块	文件数	功能定位	关键特征
sql/api	26	公共抽象 API 层，定义 SparkSession/Dataset/Column 等接口	抽象类，Classic/Connect 双实现
sql/catalyst	350+	查询优化框架：SQL 解析 → 语义分析 → 逻辑优化	TreeNode 统一树结构，规则化转换
sql/core	200+	经典执行引擎：物理计划生成 → 代码生成 → 分布式执行	SparkPlanner 策略模式，Codegen

模块	文件数	功能定位	关键特征
sql/hive	33	Hive Metastore 集成，Hive 表/UDF/SerDe 支持	类加载隔离，多版本兼容
sql/connect	67	远程客户端-服务端架构，gRPC 协议通信	Protobuf 序列化，语言无关
sql/pipelines	64	声明式数据流管道（类似 Delta Live Tables）	数据流图，拓扑排序执行

第二章 端到端查询执行流程

当用户调用 `spark.sql("SELECT ...")` 或通过 `DataFrame API` 构建查询时，Spark SQL 会经历一系列精心设计的处理阶段。这个流水线是理解整个 Spark SQL 架构的核心。



图 2-1: Spark SQL 端到端查询执行流水线 (10 个阶段)

2.1 各阶段详解

阶段	职责	核心说明
Parser	SQL 解析	ANTLR4 词法/语法分析 → AstBuilder → Unresolved LogicalPlan
Analyzer	语义分析	60+ 条规则 FixedPoint 迭代：解析表/列/函数/类型
Optimizer	逻辑优化	50+ 条规则：谓词下推、列裁剪、常量折叠、Join 重排
SparkPlanner	物理计划	18 个 Strategy 将 LogicalPlan → SparkPlan
Preparation	执行准备	14 条 rules：InsertAQE、EnsureRequirements、Codegen
Codegen	代码生成	融合算子为单 Java 方法，消除虚函数调用
AQE	自适应执行	运行时统计 → 动态调整分区/Join 策略/倾斜优化
Execution	物理执行	通过 Spark Core RDD 引擎分布式执行

第三章 Catalyst 优化框架核心设计

Catalyst 的设计精髓在于三个核心抽象：统一的树结构（TreeNode）、规则化的转换引擎（RuleExecutor）和不可变的函数式转换。

3.1 核心类继承体系

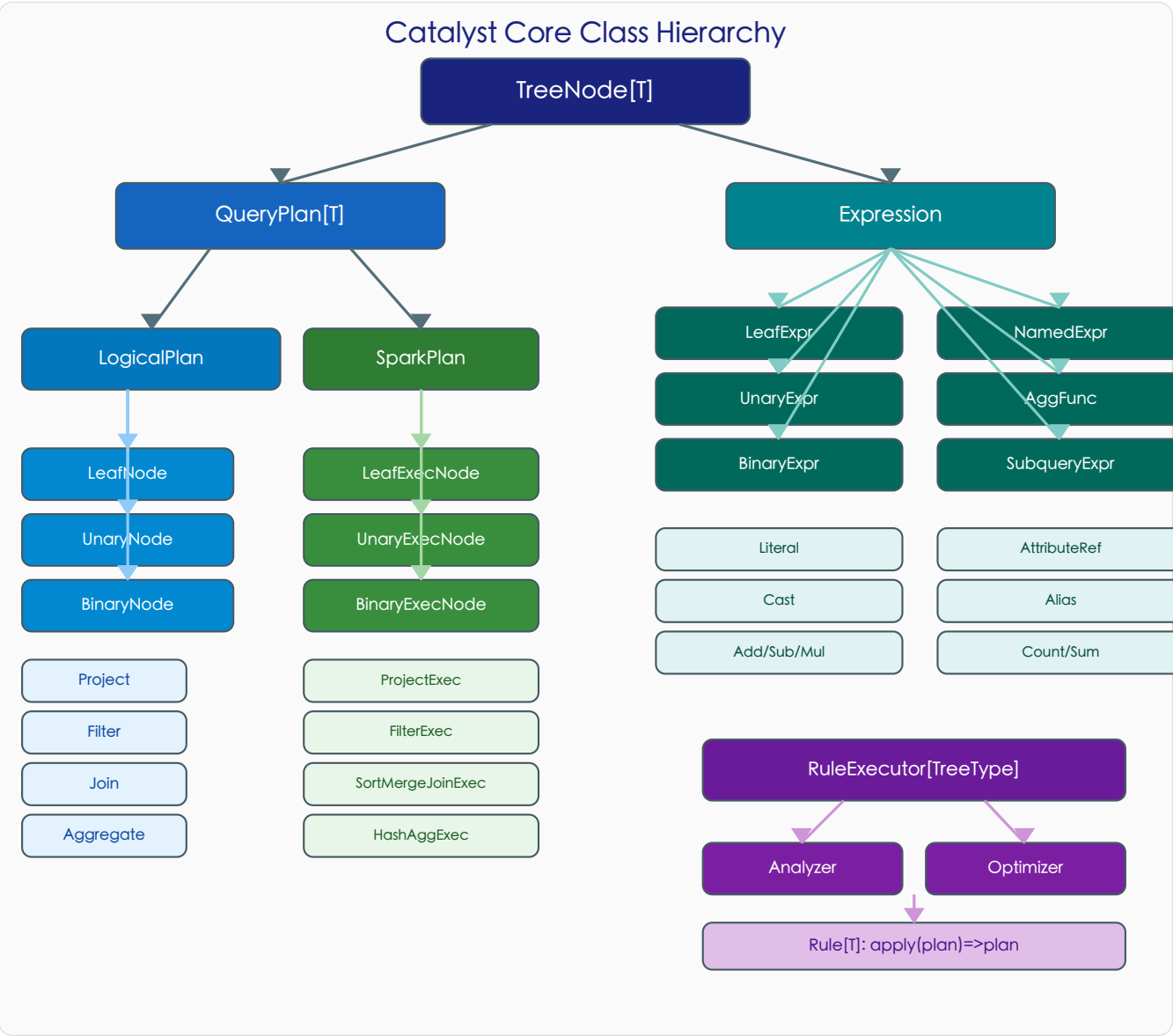


图 3-1: Catalyst 核心类继承体系

3.2 TreeNode —— 万物之基

TreeNode 是 Catalyst 整个树形结构的根基类。所有逻辑计划、物理计划和表达式都继承自它，提供不可变的树结构操作 API。

方法	返回类型	说明
transform(rule)	TreeNode	递归应用转换规则（默认 top-down）
transformDown/Up	TreeNode	前序/后序递归转换
transformWithPruning	TreeNode	带 BitSet 剪枝的转换（性能关键）
foreach/foreachUp	Unit	前序/后序遍历
collect/collectFirst	Seq/Option	收集满足条件的节点
withNewChildren	TreeNode	创建具有新子节点的副本
fastEquals	Boolean	快速相等判断（先引用比较再值比较）

3.3 RuleExecutor —— 规则引擎

概念	说明	示例
Rule[T]	单条转换规则：apply(plan) => plan	ConstantFolding, PushDownPredicates
Batch	一组规则 + 执行策略	Batch("Opt", FixedPoint(100), rules...)
Once	规则只执行一次	Finish Analysis
FixedPoint(n)	迭代直到不变或达 n 次	Resolution 批次

3.4 性能优化机制

- TreePattern BitSet 剪枝：每个节点维护 BitSet 记录子树包含的模式类型，transform 时跳过不匹配的子树
- RuleId 无效规则剪枝：记录已证明无效的规则 ID，后续迭代跳过该规则
- 不可变树 + 结构共享：只复制变化路径上的节点，未变化子树直接复用引用
- fastEquals 快速比较：先引用比较，大多数情况避免深度值比较

第四章 QueryExecution 执行阶段详解

QueryExecution

是查询生命周期的核心管理者，通过惰性求值逐步将逻辑计划转化为可执行的物理计划。

LazyTry

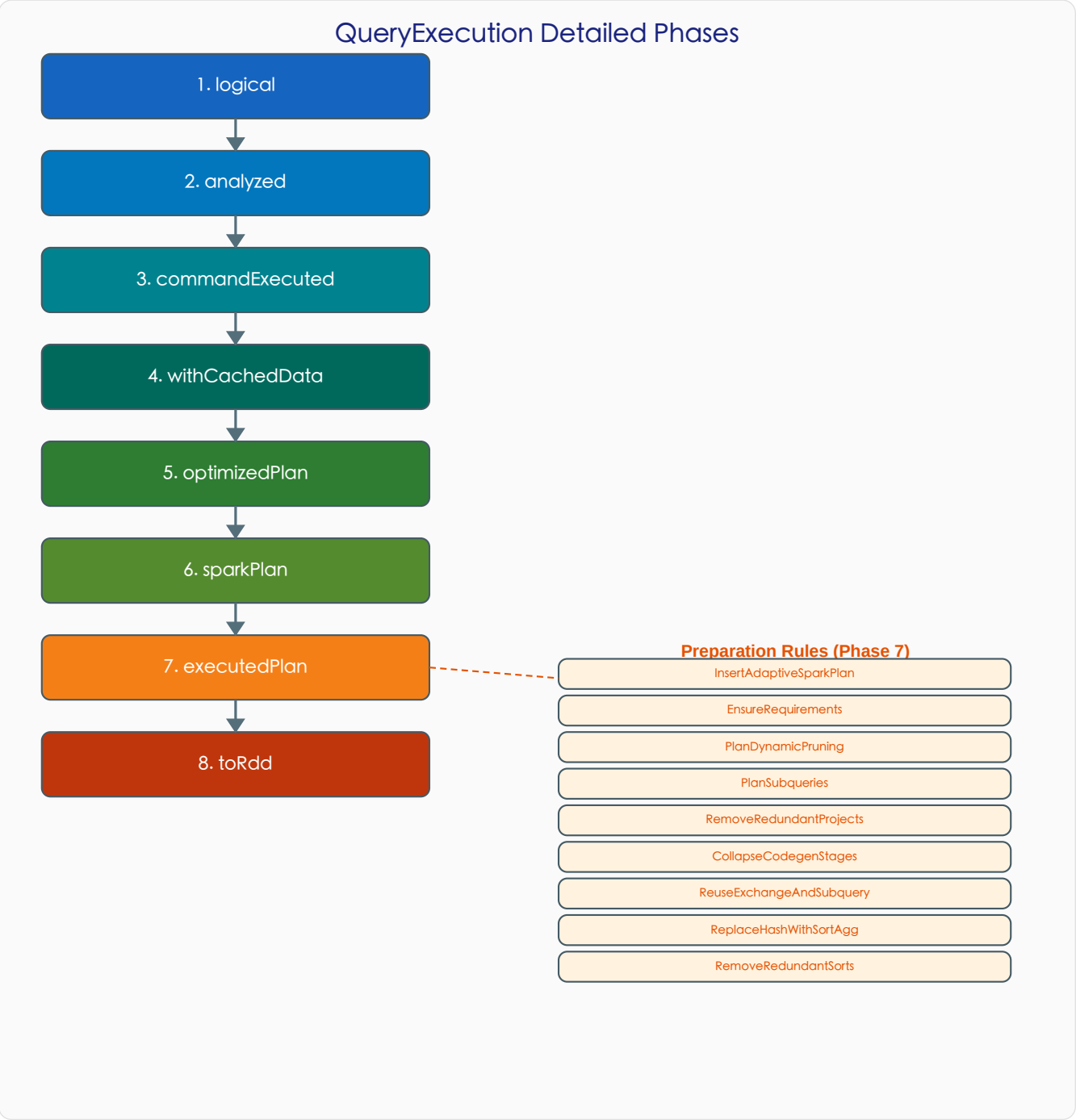


图 4-1: QueryExecution 8 个执行阶段与 Preparation Rules

4.1 Preparation Rules 详解

规则	说明
InsertAdaptiveSparkPlan	在计划根部插入 AQE 节点
EnsureRequirements	插入 Shuffle/Sort 满足数据分布和排序需求
PlanDynamicPruningFilters	运行时动态分区裁剪
PlanSubqueries	为子查询生成独立物理计划
CollapseCodegenStages	折叠连续 codegen 算子为 WholeStageCodegenExec
ReuseExchangeAndSubquery	复用相同的 Exchange 和子查询结果
ReplaceHashWithSortAgg	排序数据上将 HashAgg 替换为 SortAgg
RemoveRedundantSorts	移除已满足排序需求的冗余 Sort

第五章 物理计划生成与 Join 策略选择

5.1 SparkPlanner 策略列表

SparkPlanner 通过策略模式将逻辑计划节点转换为物理计划节点。18 个策略按优先级依次尝试匹配。

优先级	策略	说明
1	extraStrategies	用户自定义策略
2-3	LogicalQueryStageStrategy	AQE 已物化 Stage
4	PythonEvals	Python UDF 评估
5-7	DataSourceV2/File/DataSource	数据源策略
8	SpecialLimits	Limit 优化 (TakeOrderedAndProject)
9	Aggregation	Hash/Sort/Object 聚合
10-11	Window / WindowGroupLimit	窗口函数
12	JoinSelection	Join 策略选择 (最复杂)
13	InMemoryScans	缓存表扫描
14	BasicOperators	兜底：Project/Filter/Sort/Union 等

5.2 Join 策略决策流程

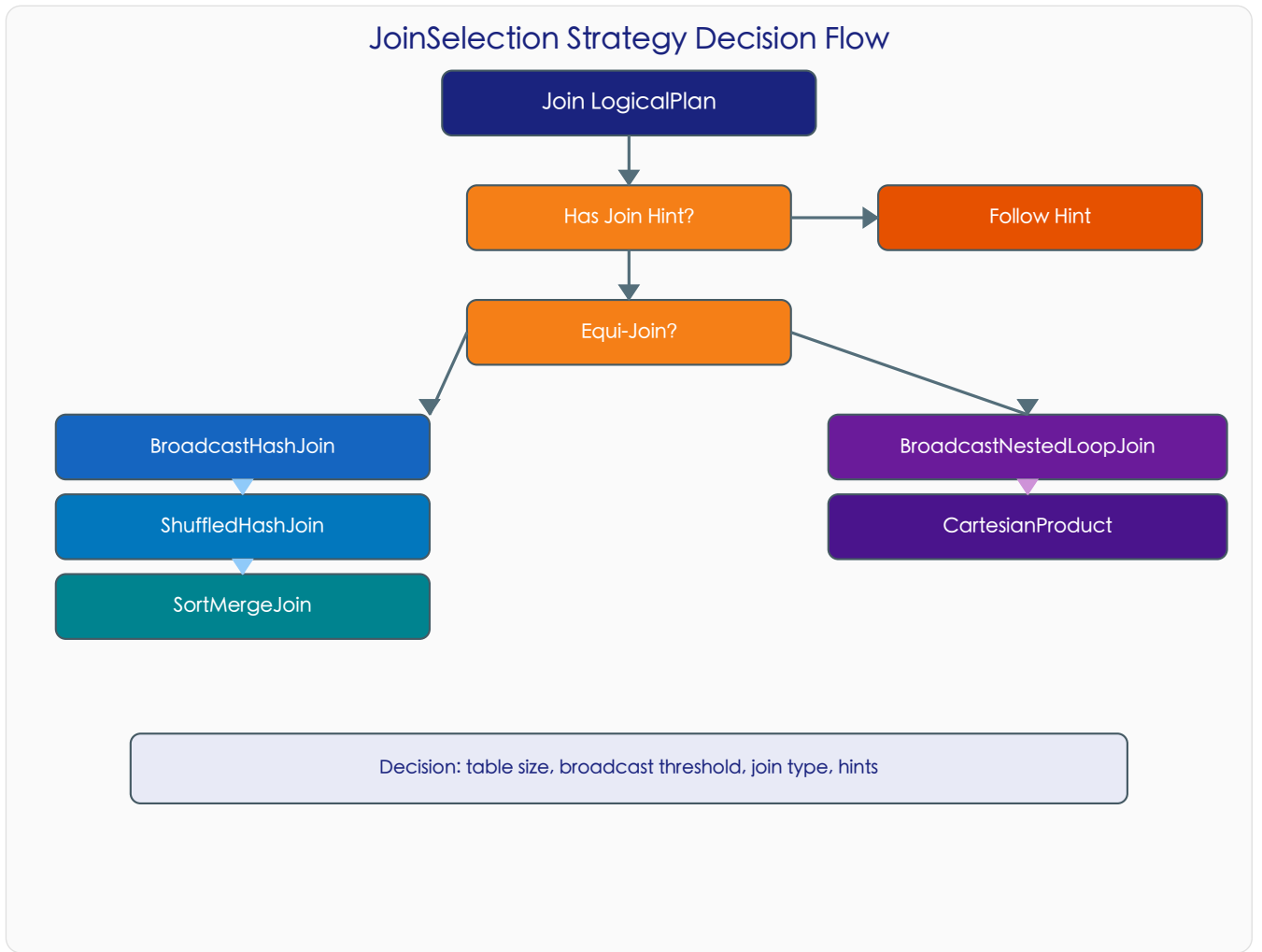


图 5-1: Join 策略选择决策流程

5.3 Join 物理算子对比

算子	原理	适用条件	性能
BroadcastHashJoin	广播小表到所有 Executor，本地 Hash	小表<10MB	$O(n)$, 最快
ShuffledHashJoin	Shuffle 后分区内 Hash Join	一侧明显较小	较快
SortMergeJoin	Shuffle+排序后归并连接	两侧都很大	$O(n \log n)$, 稳定
BroadcastNLJ	广播+嵌套循环	非等值Join+小表	$O(n*m)$
CartesianProduct	笛卡尔积	无Join条件	$O(n*m)$, 最慢

第六章 WholeStageCodegen 全阶段代码生成

WholeStageCodegen 将连续支持 codegen 的算子融合为单个 Java 方法，消除火山模型的虚函数调用开销，使 CPU 流水线利用率最大化。

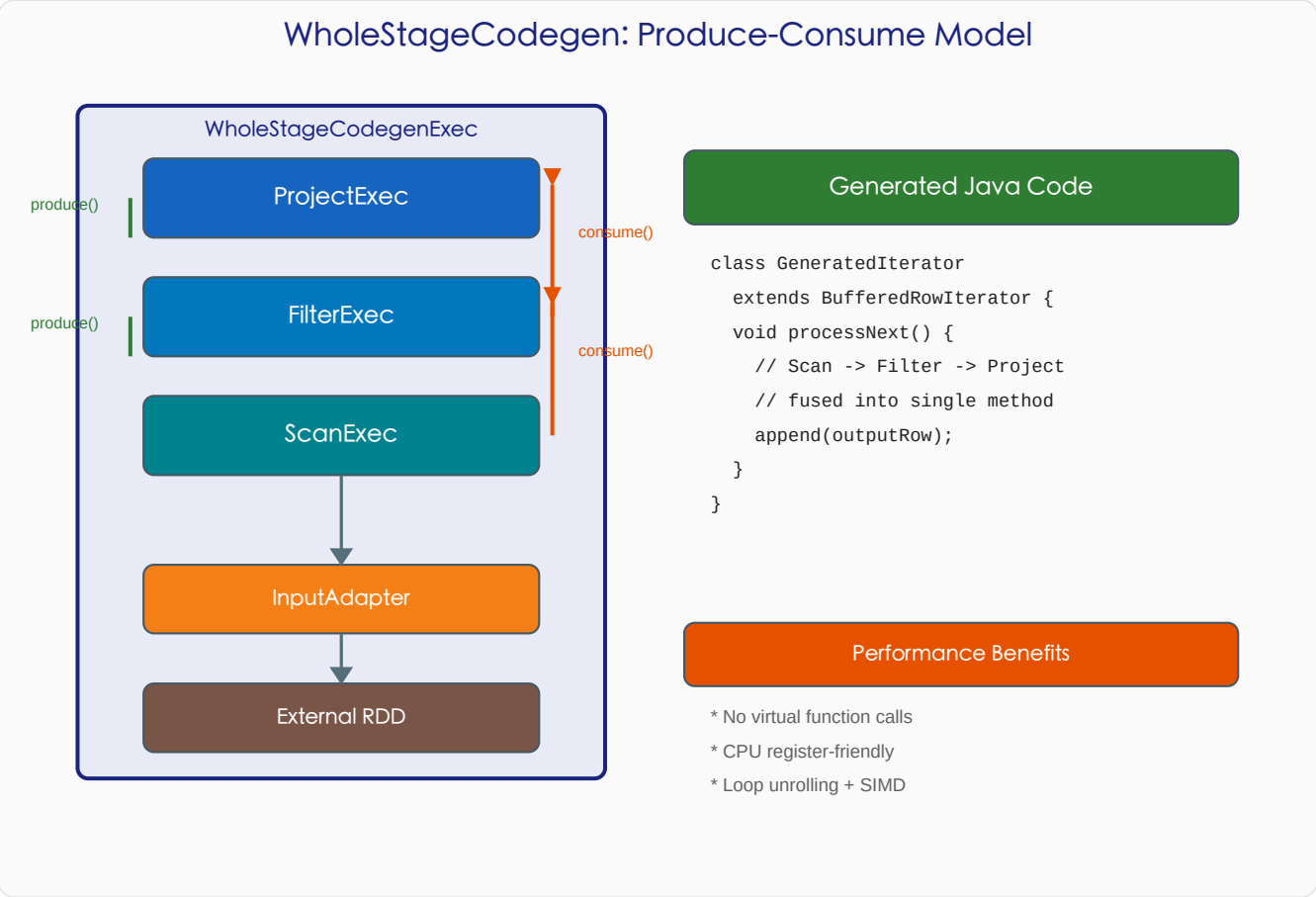


图 6-1: Produce-Consume 模型与代码生成

6.1 CodegenSupport 接口

方法	返回	说明
produce(ctx,parent)	String	生产者：生成数据产生代码
consume(ctx,vars,row)	String	消费者：向父节点传递数据
doProduce(ctx)	String	子类实现的生产逻辑
doConsume(ctx,input,row)	String	子类实现的消费逻辑
inputRDDs()	Seq[RDD]	叶子节点的输入 RDD
supportCodegen	Boolean	是否支持代码生成

第七章 AQE 自适应查询执行

AQE 在运行时根据实际数据统计动态调整查询计划。核心思想：将执行分为多个 Stage，每个 Stage 完成后收集统计，重新优化后续 Stage。

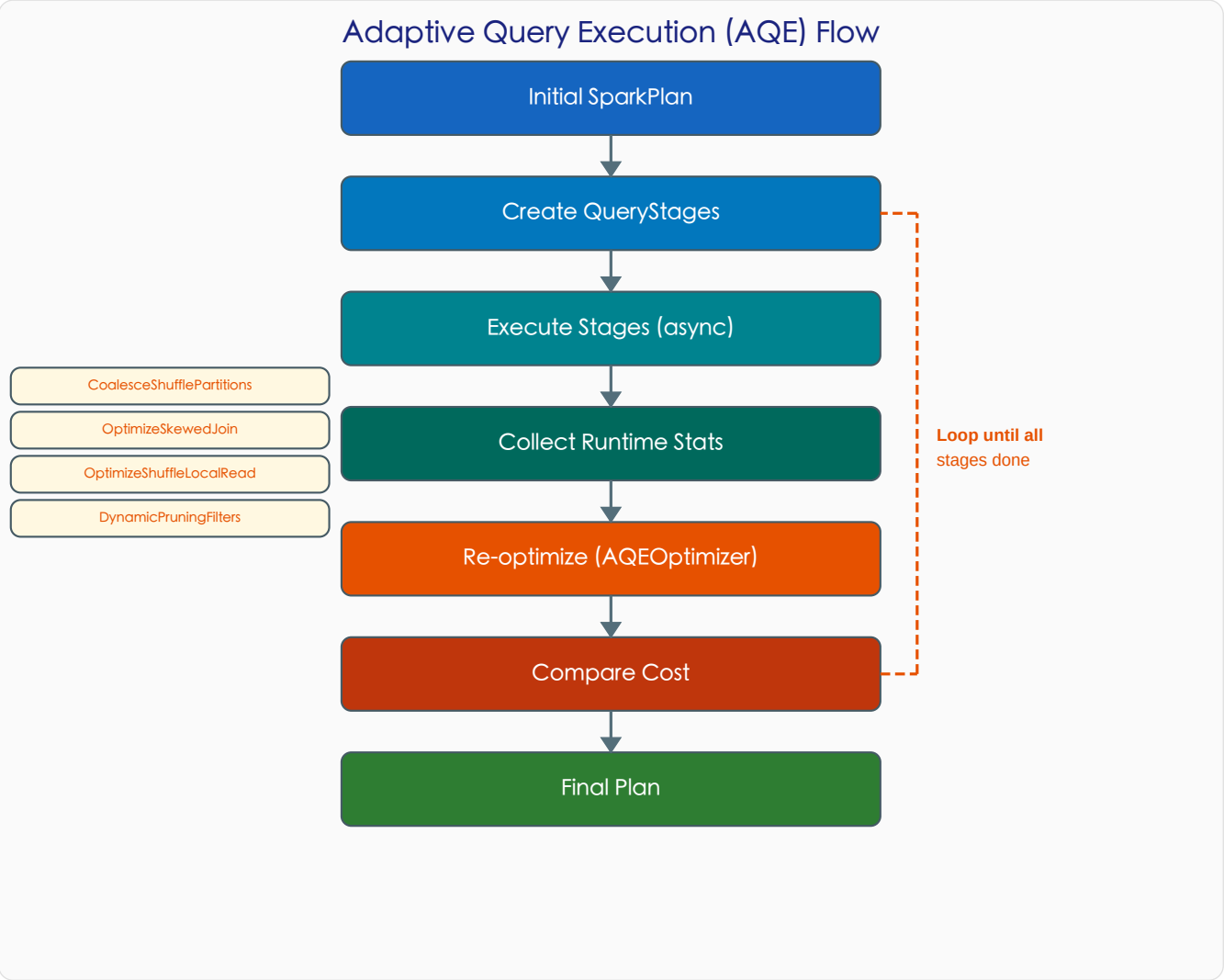


图 7-1: AQE 执行循环流程

7.1 AQE 核心优化规则

规则	功能	原理
CoalesceShufflePartitions	合并过小 Shuffle 分区	基于 advisoryPartitionSizeInBytes
OptimizeSkewedJoin	倾斜 Join 优化	拆分倾斜分区为多个子任务
OptimizeShuffleWithLocalRead	本地读取替代 Shuffle	同 Executor 直接读取
DynamicPruningFilters	运行时分区裁剪	基于 Broadcast 结果过滤

第八章 Spark Connect 远程架构

Spark Connect 实现客户端-服务端解耦，轻量级客户端通过 gRPC 远程提交查询。客户端无需 Spark 依赖，支持 Python/Scala/Java 等多语言。

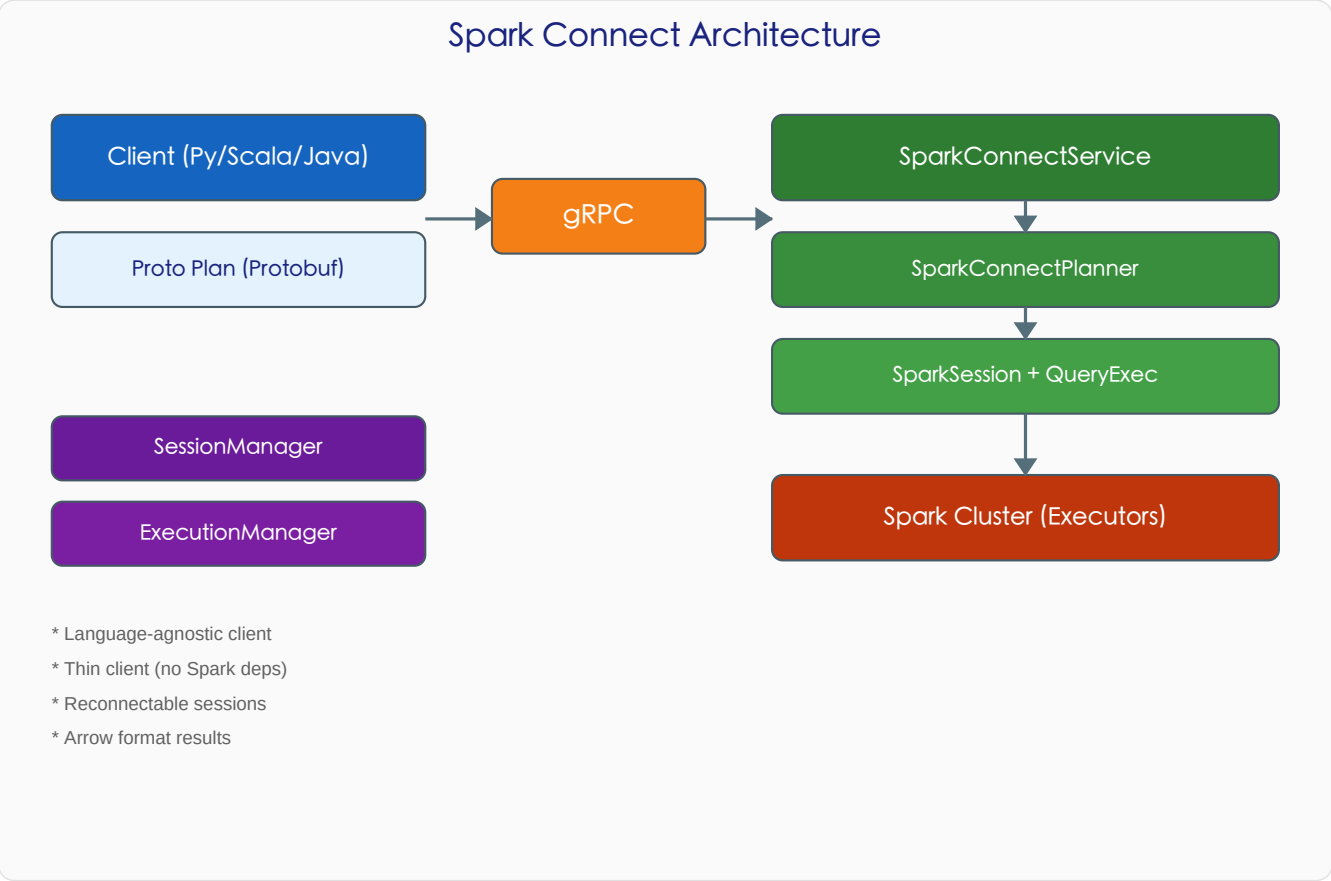


图 8-1: Spark Connect 架构

8.1 核心组件

组件	职责	说明
SparkConnectService	gRPC 服务入口	处理 executePlan/analyzePlan/config 等 RPC
SparkConnectPlanner	计划转换器(170KB)	Proto Relation → Catalyst LogicalPlan
SessionHolder	会话状态	管理 SparkSession、DataFrame 缓存等
ExecuteHolder	执行状态	请求/job tag/可重连/结果分块
SessionManager	全局会话管理	创建/获取/关闭/定时清理会话
ExecutionManager	全局执行管理	跟踪所有执行状态，墓碑缓存

第九章 Hive 集成与 Pipelines 模块

9.1 Hive 集成模块

sql/hive 模块负责与 Hive Metastore 的集成，通过类加载隔离支持多版本 Hive 兼容。

核心类	说明
HiveExternalCatalog	ExternalCatalog 的 Hive 实现，所有元数据操作委托给 HiveClient
HiveClient / HiveClientImpl	与 Hive Metastore 交互的接口和实现（反射调用 Hive API）
IsolatedClientLoader	用独立 URLClassLoader 加载指定版本 Hive jar，实现类加载隔离
HiveSessionStateBuilder	构建 Hive 感知的 SessionState，注入 Hive UDF 和优化规则
HiveTableScanExec	Hive 表扫描物理算子，支持列裁剪和分区裁剪
TableReader	读取 Hive 表数据产出 RDD[InternalRow]

Hive Metastore 交互链路

SparkSession → HiveSessionStateBuilder → HiveSessionCatalog → HiveExternalCatalog → HiveUtils.newClientForMetadata() → IsolatedClientLoader → HiveClientImpl → HiveMetaStoreClient → Thrift RPC → Hive Metastore 服务

9.2 Pipelines 模块

sql/pipelines 实现声明式数据流管道框架，用户声明数据流图，系统自动处理依赖解析和拓扑排序执行。

核心类	说明
DataflowGraph	核心图结构，管理 Flow/Table/Sink/View 及其关系
Flow + FlowFunction	数据流节点，FlowFunction 是用户查询 lambda
PipelineExecution	执行入口：resolveGraph → materialize → execute
TriggeredGraphExecution	按拓扑顺序执行所有 Flow，支持失败重试
DatasetManager	在 Catalog 中物化表，处理 Schema 合并
SqlGraphRegistrationContext	解析 SQL DDL 注册图节点

第十章 核心类设计总结

10.1 SparkSession —— 程序入口

SparkSession 是 Spark SQL 的编程入口，内部持有 SharedState（跨 Session 共享的 CacheManager/ExternalCatalog）和 SessionState（会话级的 Analyzer/Optimizer/Planner/SQLConf）。

成员	说明
sharedState	共享状态：CacheManager, ExternalCatalog, StatusTracker
sessionState	会话状态：Analyzer, Optimizer, Planner, SQLConf
sql(sqlText)	执行 SQL → parsePlan → executePlan → Dataset
read → DataFrameReader	数据源读取：json/csv/parquet/jdbc
catalog	元数据 Catalog：数据库/表/函数管理

10.2 Dataset/DataFrame —— 核心数据抽象

Dataset[T] 是 Spark SQL 最核心的数据抽象，DataFrame 即 Dataset[Row]。每个 Dataset 内部持有 QueryExecution，所有 Transformation 都是惰性的，Action 触发实际执行。

API 类别	方法	说明
Transformation	select/filter/join/groupBy/agg/sort/union	惰性转换，构建逻辑计划
Action	collect/count/show/take/foreach/write	触发执行，返回结果
I/O	read/write/save/insertInto	数据读写
Typed	map/flatMap/mapPartitions	强类型转换

10.3 SparkPlan —— 物理计划体系

类别	具体算子	说明
基础算子	ProjectExec, FilterExec, SortExec, UnionExec	基础关系代数操作
Join	BroadcastHashJoin, SortMergeJoin, ShuffledHashJoin	多种 Join 实现
聚合	HashAggregateExec, SortAggregateExec, ObjectHashAggExec	三种聚合策略
Exchange	ShuffleExchangeExec, BroadcastExchangeExec	数据重分布
代码生成	WholeStageCodegenExec, InputAdapter	全阶段代码融合
自适应	AdaptiveSparkPlanExec, QueryStageExec	AQE 运行时优化
数据源	DataSourceScanExec, InMemoryTableScanExec	数据读取

10.4 Expression —— 表达式体系

属性	类型	说明
foldable	Boolean	是否可常量折叠
deterministic	Boolean	相同输入是否始终相同输出
nullable	Boolean	结果是否可能为 null
dataType	DataType	结果数据类型
eval(row)	Any	解释执行求值
genCode(ctx)	ExprCode	代码生成 (WholeStageCodegen)
canonicalized	Expression	规范化形式 (用于语义相等判断)

第十一章 总结与架构评价

11.1 Spark SQL 架构的五大设计精髓

- 统一的树结构：TreeNode 作为万物之基，LogicalPlan/SparkPlan/Expression 共用相同的遍历和转换 API，极大简化了框架设计
- 规则化转换引擎：每个分析/优化步骤封装为独立的 Rule，通过 RuleExecutor 的 Batch+Strategy 机制组合执行，易于扩展和调试
- 不可变树 + 函数式转换：每次 transform 返回新树，保证线程安全和可推理性。结构共享机制保证性能
- 策略模式的物理计划生成：SparkPlanner 的 18 个策略按优先级匹配，支持用户自定义扩展策略
- 运行时自适应优化：AQE 突破静态优化的局限，利用运行时统计信息动态调整执行策略，显著提升复杂查询性能

11.2 关键数据流总结

维度	描述
输入	SQL 字符串 / DataFrame API 链式调用
解析输出	Unresolved LogicalPlan (含 UnresolvedRelation/Attribute)
分析输出	Resolved LogicalPlan (所有引用已绑定)
优化输出	Optimized LogicalPlan (下推/裁剪/折叠后)
物理计划	SparkPlan (含具体物理算子选择)
可执行计划	ExecutedPlan (满足分布/排序/codegen 需求)
最终执行	RDD[InternalRow] → Spark Core 分布式执行

11.3 代码规模统计

模块	Scala 文件数	核心类数	最大文件	关键复杂度
catalyst	350+	~500	AstBuilder(269KB)	Analyzer(4252行), Optimizer(2944行)
core	200+	~300	Dataset(87KB)	SparkStrategies(53KB), QueryExecution(815行)
hive	33	~50	HiveClientImpl(64KB)	类加载隔离+多版本兼容
connect	67	~100	SparkConnectPlanner(170KB)	Proto ↔ Catalyst 全量转换
pipelines	64	~80	SqlGraphRegCtx(689行)	图拓扑排序+流批统一